CDB87F8F-569E-4092-8CBB-0C026AC23351

XXX_DERS_PROGRAMI

Sayısal Yöntemler: Hata Analizi ve Sayısal Gösterimler

#ders #donem3 #sayisal_yontemler

1. Hata Tanımları (Error Definitions)

Sayısal yöntemlerde analitik (tam) çözümü elde etmek çoğu zaman mümkün veya pratik değildir; bu nedenle yaklaşık (sayısal) çözümler aranır. Bir yaklaşık değerin gerçeğe ne kadar yakın olduğunu ölçmek için hata tanımları kullanılır.

- x: Gerçek (tam) değer
- x^*: Yaklaşık (hesaplanan) değer

1.1. Mutlak Hata (Absolute Error)

Gerçek değer ile yaklaşık değer arasındaki farkın mutlak büyüklüğüdür:

E_t = |x - x^*|

(Bazı kaynaklarda işaret bilgisini korumak için doğrudan $x - x^$ olarak da tanımlanır.)*

- **Birim:** Gerçek değer ile aynı birime sahiptir (metre, saniye vb.).
- **Eksik yönü:** Sayının büyüklüğü hakkında fikir vermez. Örneğin 1000 metrede 1 cm hata ile 2 cm'de 1 cm hata aynı mutlak hatayı verir (1 cm), fakat önem dereceleri çok farklıdır.

1.2. Bağıl Hata (Relative Error)

Mutlak hatanın gerçek değere oranlanmasıyla elde edilir. Boyutsuz (birimsiz) bir büyüklüktür ve hatanın büyüklüğünü sayının ölçeğine göre değerlendirir:

\epsilon_t = \frac{|x - x^*|}{|x|} \quad (x \neq 0)

1.3. Yüzde Bağıl Hata (Percentage Relative Error)

Bağıl hatanın yüzdelik olarak ifade edilmiş halidir:

\%\epsilon_t = \leftl \frac{x - x^*}{x} \rightl \times 100\%

1.4. Yaklaşık / İteratif Bağıl Hata ve Durma Kriteri K

Gerçek değer (x) çoğu problemde baştan bilinmez. Bu durumda iteratif (ardışık yaklaşımlı) yöntemlerde iki ardışık adım arasındaki değişim incelenir:

\epsilon_a = \leftl \frac{x_{yeni} - x_{eski}}{x_{yeni}} \rightl \times 100\% = \leftl \frac{x_{i+1} - x_i}{x_{i+1}} \rightl \times 100\%

- **Durma Kriteri:** Önceden belirlenen bir tolerans sınırına (\epsilon_s) ulaşıldığında iterasyon sonlandırılır:
  \epsilon_a < \epsilon_s
- Scarborough kriterine göre n anlamlı basamağa kadar doğruluk için:
  \epsilon_s = (0.5 \times 10^{2-n})\% \quad \text{K}

2. Hata Kaynakları (Sources of Errors)

Sayısal hesaplamalarda nihai sonuca etki eden başlıca hata kaynakları şunlardır:

1. **Modelleme Hataları (Modeling Errors):**
   - İncelenen fiziksel problemin matematiksel modele aktarılması sırasında yapılan basitleştirmeler, ihmaller ve kabullerdir (Örn: Sürtünmenin veya hava direncinin ihmal edilmesi, lineer olmayan durumların lineer kabul edilmesi).
2. **Veri ve Ölçüm Hataları (Data / Measurement Errors):**
   - Modele girdi olarak verilen parametrelerin ve ölçümlerin fiziksel sınırlarından veya cihaz hassasiyetinden kaynaklanan hatalardır (Örn: Bir sensörün sıcaklığı \pm 0.1^\circ\text{C} sapmayla ölçmesi).
3. **Kesme Hataları (Truncation Errors):**
   - Sonsuz bir matematiksel sürecin (sonsuz seriler, limitler, türev/integral tanımları) sonlu sayıda işlemle ifade edilmesi sonucu ortaya çıkan kuramsal hatalardır (Örn: Taylor serisinin ilk birkaç terimden sonra kesilmesi).
4. **Yuvarlama Hataları (Round-off Errors):**
   - Sayıların bilgisayar belleğinde veya kağıt üzerinde sonlu sayıda basamakla

saklanması ve temsil edilmesinden kaynaklanan hatalardır (Örn: 1/3 = 0.333333... sayısının 0.3333 olarak saklanması).

5. **İnsan ve Programlama Hataları (Blunders / Gross Errors) K:**
    - Kodlama hataları, formüllerin koda yanlış aktarılması, algoritma mantık hataları veya veri giriş yanlışlıklarıdır.

3. Hata Çeşitleri (Types of Errors)

Sayısal yöntemler dersinde özellikle üzerinde durulan hata çeşitleri:

3.1. Yuvarlama Hatası (Round-off Error)

Sayıların sonlu basamak kısıtından dolayı oluşur. İki temel şekilde gerçekleştirilir:

- **Kırpma / Kesme (Chopping):** Belirli bir basamaktan sonraki tüm hanelerin doğrudan atılmasıdır.
    - Örn: 3.141592... sayısı 4 anlamlı basamakla kırpılırsa \rightarrow 3.141
- **Yuvarlama (Rounding):** Atılacak ilk basamağın büyüklüğüne göre son basamağın ayarlanmasıdır.
    - Eğer atılan hane \ge 5 ise son basamak 1 artırılır, < 5 ise aynen kalır.
    - Örn: 3.141592... sayısı 4 anlamlı basamakla yuvarlanırsa \rightarrow 3.142

3.2. Kesme Hatası (Truncation Error)

Matematiksel fonksiyon ve operatörlerin yaklaşım formülleriyle değiştirilmesi sonucu doğar.

- **Taylor Serisi Kesmesi:**
  f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \dots + R_nBuradaki R_n terimi kesme hatasını temsil eder ve adım boyutu h'a bağlıdır (O(h) veya O(h^2)).

3.3. Hata Yayılımı (Propagation of Error) K

Girdilerdeki küçük hataların aritmetik işlemler (+, -, \times, \div) sonucunda birikerek büyümesidir.

- **Yok Olma Hatası (Catastrophic Cancellation / Sayısal Kararsızlık):** Birbirine çok yakın iki büyük sayının birbirinden çıkarılması durumunda, anlamlı basamakların çoğu sıfırlanır ve geriye sadece yuvarlama hatası kalır:
  \Delta = a - b \quad (a \approx b)Bu durum hesaplamada çok büyük bağıl hatalara yol

açar.

4. Normalleştirilmiş Bilimsel Gösterim (SEF - Scientific Exponential Form / Standart E-Formatı)

Bilgisayarlar ve bilimsel hesap makineleri, çok büyük veya çok küçük sayıları sonlu bellek alanında tutabilmek için **Kayan Noktalı Sayı (Floating-Point)** ve **Normalleştirilmiş Bilimsel Gösterim** yapısını kullanır.

4.1. Tanım ve Matematiksel Biçim

Bir sayının **normalleştirilmiş bilimsel gösterimi** (SEF), virgülden önce sıfırdan farklı tek bir basamak (veya sayısal analiz kaynaklarına göre virgülden hemen sonra sıfırdan farklı ilk basamak) bulunacak şekilde yazılmasıdır:

Standart Bilimsel Form (Standart E-Formatı):

x = \pm m \times 10^e \quad \text{veya bilgisayar gösterimiyle} \quad \pm m \text{E} eBurada:

- \pm: Sayının **işareti (sign)**
- m: **Mantis (Mantissa / Significand)**, 1 \le |m| < 10
- e: **Üs (Exponent)**, tam sayıdır (e \in \mathbb{Z})

Sayısal Analiz / Mantis Kesirli Formu (Alternatif Normalleştirilmiş Biçim):

Birçok sayısal yöntem kitabında normalleştirilmiş kayan noktalı form şu şekilde tanımlanır: x = \pm 0.d_1 d_2 d_3 \dots d_k \times 10^n \quad (d_1 \neq 0)Burada 0.1 \le |m| < 1'dir ve virgülden sonraki ilk hane (d_1) sıfır olamaz.

4.2. Örnek Dönüşümler

- 5432.1 \longrightarrow +5.4321 \times 10^3 \quad (\text{SEF: } 5.4321\text{E}+3)
  - *Mantis kesirli formda:* +0.54321 \times 10^4
- -0.000482 \longrightarrow -4.82 \times 10^{-4} \quad (\text{SEF: } -4.82\text{E}-4)
  - *Mantis kesirli formda:* -0.482 \times 10^{-3}

4.3. Kaymalı Normalleştirilmiş Sayı Gösterimi (Biased Normalized Numbers / Excess-K)

Normalleştirilmiş kayan noktalı sayılarda üs (e) hem pozitif (büyük sayılar için) hem de negatif (küçük sayılar için) değer alabilir. Bilgisayarlar bu üsleri işaret bitiyle veya 2'ye tümleyenle saklamak yerine, donanım düzeyinde işlem kolaylığı sağlamak amacıyla pozitif bir **kayma**

değeri (Bias) ekleyerek saklarlar.

1. Kayma (Bias) ve Karakteristik Mantığı:

- k: Üs (exponent) alanına ayrılan bit sayısı.
- **Kayma Değeri (\text{Bias}):**
  \text{Bias} = 2^{k-1} - 1*(Örn: 8-bit üs için \text{Bias} = 2^{8-1} - 1 = 127; 11-bit üs için \text{Bias} = 2^{11-1} - 1 = 1023)*
- **Kaymalı Üs (E - Karakteristik / Biased Exponent):**
  E = e + \text{Bias} \iff e = E - \text{Bias}Böylece kaydedilen E değeri daima işaretsiz pozitif bir tam sayı (0 \le E \le 2^k - 1) olur.

2. Normalleştirme Kuralı ve Gizli Bit (Hidden / Implicit Bit):

İkili sistemde normalleştirilmiş bir sayıda virgülün solundaki basamak sıfır olamaz, bu yüzden kesinlikle 1'dir:
x = (-1)^s \times (1.f)_2 \times 2^{E - \text{Bias}}

- Virgülden önceki 1 donanımda **fiziksel olarak saklanmaz** (**Gizli Bit / Hidden Bit**). Bu sayede mantis alanında fazladan 1 bitlik hassasiyet kazanılır. Belleğe sadece virgülden sonraki kesir (f) yazılır.

3. Neden Kaymalı (Biased) Gösterim Kullanılır?

1. **Donanımsal Hız ve Hiyerarşik Karşılaştırma:** İki kayan noktalı sayının büyüklüğü karşılaştırılırken karmaşık işaretli aritmetik devrelerine gerek kalmaz; standart ve hızlı işaretsiz tamsayı karşılaştırıcıları (unsigned integer comparators) kullanılır.
2. **Özel Durumların Kolay Yönetimi:**
   - E = 0 ve f = 0 \implies \pm 0
   - E = 0 ve f \neq 0 \implies **Normalleştirilmemiş (Subnormal / Denormalized) Sayılar:** Gizli bit 0 kabul edilir (0.f), sıfıra kademeli yaklaşımı (gradual underflow) sağlar.
   - E = 2^k - 1 ve f = 0 \implies \pm \infty (Sonsuzluk / Taşma)
   - E = 2^k - 1 ve f \neq 0 \implies \text{NaN} (Not a Number - Tanımsız)

4.4. IEEE 754 Standardı Format Özeti K

Format	Toplam Bit	İşaret (s)	Kaymalı Üs (E)	Bias	Kesir/M (f

Single Precision (Tek Duyarlık / Float)	32 bit	1 bit	8 bit (E \in [1, 254])	**127**	23 bit gizli = 2
Double Precision (Çift Duyarlık / Double)	64 bit	1 bit	11 bit (E \in [1, 2046])	**1023**	52 bit gizli = 5

4.5. Makine Epsilonu, Taşma ve Alt Taşma K

- **Makine Epsilonu (\epsilon_{makine}):** 1 + \epsilon > 1 eşitsizliğini sağlayan en küçük pozitif \epsilon sayısıdır. Bilgisayarın ayırt edebileceği bağıl hassasiyet sınırını gösterir (\text{float} için 2^{-23} \approx 1.19 \times 10^{-7}, \text{double} için 2^{-52} \approx 2.22 \times 10^{-16}).
- **Taşma (Overflow):** Sayının sistemin saklayabileceği maksimum üs değerini aşmasıdır (+\infty / -\infty).
- **Alt Taşma (Underflow):** Sayının sistemin temsil edebileceği en küçük pozitif değerden daha küçük olup sıfıra yuvarlanmasıdır.

5. Adım Adım Çözümlü Örnek Sorular K

Soru 1: Mutlak, Bağıl ve Yüzde Bağıl Hata Hesabı

Soru: Bir elektrik devresindeki direncin gerçek değeri R = 450 \ \Omega olarak bilinmektedir. Yapılan ölçümde R^* = 445.5 \ \Omega bulunmuştur. Mutlak hatayı, bağıl hatayı ve yüzde bağıl hatayı bulunuz.

Çözüm:

1. **Mutlak Hata:**
   E_t = |R - R^*| = |450 - 445.5| = 4.5 \ \Omega
2. **Bağıl Hata:**

\epsilon_t = \frac{|R - R^*|}{|R|} = \frac{4.5}{450} = 0.01

3. **Yüzde Bağıl Hata:**
   \%\epsilon_t = \epsilon_t \times 100\% = 0.01 \times 100\% = \%1.0

Soru 2: SEF, Kırpma (Chopping) ve Yuvarlama (Rounding)

Soru: x = \frac{2}{7} sayısını:

1. Normalleştirilmiş bilimsel gösterimde (SEF) yazınız.
2. 4 anlamlı basamak kullanarak **kırpma (chopping)** yöntemiyle yaklaşık değerini ve yüzde bağıl hatasını bulunuz.
3. 4 anlamlı basamak kullanarak **yuvarlama (rounding)** yöntemiyle yaklaşık değerini ve yüzde bağıl hatasını bulunuz.

Çözüm:
Gerçek değer: x = \frac{2}{7} = 0.2857142857...

1. **SEF (Normalleştirilmiş Bilimsel Gösterim):**
   x = 2.857142857... \times 10^{-1} \quad (\text{veya } 0.2857142857... \times 10^0)
2. **4 Anlamlı Basamakla Kırpma (Chopping):**
   - Sayı 4 basamakta kesilir: x^*_{kesme} = 2.857 \times 10^{-1} = 0.2857
   - Mutlak Hata: |0.28571428... - 0.2857| \approx 0.0000142857
   - Yüzde Bağıl Hata:
     \%\epsilon_t = \left| \frac{2/7 - 0.2857}{2/7} \right| \times 100\% = \left| 1 - \frac{0.2857 \times 7}{2} \right| \times 100\% = \left| 1 - 0.99995 \right| \times 100\% = \%0.005
3. **4 Anlamlı Basamakla Yuvarlama (Rounding):**
   1. basamağa bakılır (1 < 5), dolayısıyla yukarı yuvarlanmaz:
      x^*_{yuvarlama} = 2.857 \times 10^{-1} = 0.2857 *(Eğer 5 anlamlı basamak istenseydi: 0.285714... \rightarrow 2.8571\text{E}-1 olurdu; 6. basamak 4 olduğu için 1 kalırdı).*

Soru 3: İteratif Durma Ölçütü Örneği K

Soru: Bir kök bulma algoritmasında x_1 = 2.500 ve sonraki iterasyonda x_2 = 2.512 hesaplanmıştır. Yaklaşık bağıl hatayı (\epsilon_a) hesaplayınız. Tolerans \epsilon_s = \%0.5 ise işlem durdurulmalı mıdır?

Çözüm:
$$\epsilon_a = \left| \frac{x_2 - x_1}{x_2} \right| \times 100\% = \left| \frac{2.512 - 2.500}{2.512} \right| \times 100\% = \frac{0.012}{2.512} \times 100\% \approx \%0.4777$$

Karşılaştırma:
$$\epsilon_a = \%0.4777 < \epsilon_s = \%0.5000$$Şart sağlandığı için iterasyon **durdurulmalıdır**.

Soru 4: Kaymalı Normalleştirilmiş Sayı (IEEE 754 Single Precision) Dönüşümü K

Soru: X = -(26.625)_{10} sayısını 32-bit IEEE 754 formatında **kaymalı normalleştirilmiş** biçime dönüştürünüz ve 32-bit ikili (binary) ile onaltılı (hexadecimal) karşılığını yazınız.

Çözüm:

1. **İkili Tabana Dönüştürme:**
   - Tam kısım (26): 26 = 16 + 8 + 2 = (11010)_2
   - Kesirli kısım (0.625): 0.625 = 0.5 + 0.125 = (0.101)_2
   - Birleşik değer: (26.625)_{10} = (11010.101)_2
2. **Normalleştirme (Kayan Noktayı Kaydırma):**
   - Virgül en soldaki 1'in sağına gelecek şekilde 4 basamak sola kaydırılır:
     $$(11010.101)_2 = 1.1010101_2 \times 2^4$$
   - Buradan gerçek üs: e = 4
3. **İşaret Biti (s):**
   - Sayı negatif olduğu için: s = 1 (1 bit)
4. **Kaymalı Üs (E - Karakteristik):**
   - 32-bit için \text{Bias} = 127:
     $$E = e + \text{Bias} = 4 + 127 = 131$$
   - 131 sayısının 8-bit ikili karşılığı:
     $$131_{10} = 128 + 2 + 1 = (10000011)_2 \quad \text{(8 bit)}$$

5. **Mantis / Kesir Alanı (f - 23 bit):**
    - Virgülden önceki gizli bit (1.) atılır: 1010101
    - Toplam 23 bite ulaşmak için sağına sıfırlar eklenir:
      $f = 10101010000000000000000 \quad \text{(23 bit)}$
6. **32-Bit Birleşik Sonuç ve Hexadecimal Karşılığı:**
   $\underbrace{1}_{s} \quad \underbrace{10000011}_{E \ (8 \text{ bit})} \quad \underbrace{10101010000000000000000}_{f \ (23 \text{ bit})}$
    - 4'erli gruplayarak Onaltılı (Hex) tabana çevirelim:
        - $1100 \rightarrow \text{C}$
        - $0001 \rightarrow 1$
        - $1101 \rightarrow \text{D}$
        - $0101 \rightarrow 5$
        - $0000 \rightarrow 0$
        - $0000 \rightarrow 0$
        - $0000 \rightarrow 0$
        - $0000 \rightarrow 0$
    - **Hexadecimal Karşılığı:** $\mathbf{(\text{C1D50000})_{16}}$

Soru 5: -0.75 Sayısının Kayan Noktalı Sayı (KYS) ile İfade Edilmesi

Soru: X = -0.75 sayısını Kayan Noktalı Sayı (KYS / Floating Point) standart kaymalı normalleştirilmiş formatta (IEEE 754 Single Precision - 32 bit) ifade ediniz.

Çözüm:

1. **Sayıyı İkili (Binary) Tabana Çevirme:**
    - $0.75 = \frac{3}{4} = \frac{1}{2} + \frac{1}{4} = 2^{-1} + 2^{-2} = (0.11)_2$
    - Dolayısıyla:
      $-(0.75)_{10} = -(0.11)_2$
2. **Normalleştirme (Virgülü Kaydırma):**

- Normalleştirilmiş bilimsel biçimde virgülden önce tek bir 1 bulunmalıdır:
  $-(0.11)_2 = -1.1_2 \times 2^{-1}$
- Buradan:
  - **Gerçek üs:** e = -1
  - **Mantis (kesir kısmı):** virgülden sonraki kısım .1 (virgülün solundaki 1 gizli bittir).

3. **İşaret Biti (s):**
   - Sayı negatif olduğu için:
     $s = 1 \quad \text{(1 bit)}$
4. **Kaymalı Üs (E - Karakteristik) Hesabı:**
   - 32-bit KYS formatında kayma miktarı $\text{Bias} = 127$'dir:
     $E = e + \text{Bias} = -1 + 127 = 126$
   - 126_{10} sayısını 8-bit ikili tabana çevirelim:
     - $126 = 64 + 32 + 16 + 8 + 4 + 2$
     - $E = (01111110)_2 \quad \text{(8 bit)}$
5. **Mantis / Kesir Alanı (f - 23 bit):**
   - Gizli bit (1.) yazılmaz, sadece virgülden sonraki 1 alınır:
   - 23 bite tamamlamak için sağ tarafı sıfırlarla doldurulur:
     $f = 10000000000000000000000 \quad \text{(23 bit)}$
6. **32-Bit KYS (Kayan Noktalı Sayı) Temsili:**
   $\underbrace{1}_{s \ (1)} \quad \underbrace{01111110}_{E \ (8 \text{ bit})} \quad \underbrace{10000000000000000000000}_{f \ (23 \text{ bit})}$

LİNEER OLMAYAN DENKLEM ÇÖZÜMÜ (ROOTS OF NON-LINEAR EQUATIONS)

Mühendislik ve temel bilimlerde karşılaşılan birçok fiziksel problem $f(x) = 0$ biçiminde cebirsel (polinom) veya transandant (üstel, trigonometrik, logaritmik) denklemlere dönüşür. Bu denklemlerin çoğunun analitik (kapalı formda) kökünü bulmak imkansızdır veya pratik değildir. Bu durumlarda sayısal (iteratif) kök bulma yöntemlerine başvurulur.

Ara Değer Teoremi (Bolzano Teoremi):
$f(x)$ fonksiyonu $[a, b]$ kapalı aralığında sürekli olsun. Eğer $f(a) \cdot f(b) < 0$ ise (yani

fonksiyon uç noktalarda zıt işaretli değerler alıyorsa), (a, b) açık aralığında f(x^*) = 0 olacak şekilde **en az bir x^* kökü** vardır.

Kök bulma yöntemleri temel olarak ikiye ayrılır:

1. **Kapalı / Parantezleme Yöntemleri (Bracketing Methods):** Kökü içeren bir aralıkla başlar ve her adımda kökün etrafını daraltır. Daima yakınsar (Örn: İkiye Bölme Yöntemi, Regula Falsi).
2. **Açık Yöntemler (Open Methods):** Tek bir başlangıç tahminiyle başlar. Çok daha hızlı yakınsayabilir ancak uygun başlangıç seçilmezse ıraksayabilir (Örn: Sabit Nokta İterasyonu, Newton-Raphson, Kirişler Yöntemi).

1. İkiye Bölme Yöntemi (Bisection Method / Yarılama Yöntemi)

İkiye bölme yöntemi, Ara Değer Teoremi'ne dayanan en temel ve en güvenilir kapalı kök bulma yöntemidir.

1.1. Yöntemin Mantığı ve Algoritması

Kökü içeren [a, b] aralığı her iterasyonda tam ortasından ikiye bölünür ve kökün hangi yarıda kaldığı test edilir.

Algoritma Adımları:

1. f(a) \cdot f(b) < 0 şartını sağlayan bir [a_1, b_1] başlangıç aralığı belirlenir.
2. k. iterasyonda orta nokta (yaklaşık kök) hesaplanır:
   c_k = \frac{a_k + b_k}{2}
3. Fonksiyon değeri f(c_k) hesaplanır:
   - Eğer f(a_k) \cdot f(c_k) < 0 ise: Kök sol yarıdadır \implies a_{k+1} = a_k, \ b_{k+1} = c_k.
   - Eğer f(a_k) \cdot f(c_k) > 0 ise: Kök sağ yarıdadır \implies a_{k+1} = c_k, \ b_{k+1} = b_k.
   - Eğer f(c_k) = 0 ise: c_k tam köktür, işlem biter.
4. Durma kriteri kontrol edilir:
   - Yaklaşık bağıl hata: \epsilon_a = \leftl \frac{c_k - c_{k-1}}{c_k} \rightl \times 100\% < \epsilon_s

- veya mutlak aralık uzunluğu: \frac{|b_k - a_k|}{2} \le \text{Tolerans}
- Şart sağlanmıyorsa 2. adıma dönülür.

1.2. Hata Analizi ve Gerekli İterasyon Sayısı

Her iterasyonda aralık boyutu yarıya iner (1/2). n adım sonra aralık boyutu \frac{b-a}{2^n} olur. Maksimum mutlak hatanın \epsilon değerinden küçük olması için gereken **minimum iterasyon sayısı (n)**:

\frac{b - a}{2^n} \le \epsilon \implies 2^n \ge \frac{b - a}{\epsilon} \implies n \ge \frac{\ln(b - a) - \ln(\epsilon)}{\ln(2)} = \log_2\left(\frac{b - a}{\epsilon}\right)

- **Avantajı:** Fonksiyon sürekli olduğu ve kökü parantezlediği sürece **daima yakınsar (garantili yakınsama)**.
- **Dezavantajı:** Yakınsama hızı düşüktür (lineer yakınsama, O(1/2^n)).

İkiye Bölme Yöntemi — Çözümlü Örnekler

Örnek 1.1: Polinom Denklemi Çözümü

Soru: f(x) = x^3 - x - 2 = 0 denkleminin [1, 2] aralığındaki kökünü İkiye Bölme Yöntemi kullanarak 4 iterasyon boyunca hesaplayınız. Her adımda yaklaşık bağıl hatayı (\epsilon_a) bulunuz.

Çözüm:
Başlangıç kontrolü:

- f(1) = 1^3 - 1 - 2 = -2 < 0
- f(2) = 2^3 - 2 - 2 = 4 > 0
- f(1) \cdot f(2) = (-2)(4) = -8 < 0 (Aralıkta en az bir kök vardır).

1. İterasyon (k=1):

- a_1 = 1, \ b_1 = 2
- c_1 = \frac{1 + 2}{2} = 1.5000
- f(c_1) = f(1.5) = (1.5)^3 - 1.5 - 2 = 3.375 - 1.5 - 2 = -0.125 < 0
- f(a_1) \cdot f(c_1) = (-2) \cdot (-0.125) > 0 \implies Kök [1.5, 2] aralığındadır.
- Yeni aralık: a_2 = 1.5000, \ b_2 = 2.0000

2. İterasyon (k=2):

- $c_2 = \frac{1.5 + 2}{2} = 1.7500$
- $f(c_2) = f(1.75) = (1.75)^3 - 1.75 - 2 = 5.359375 - 1.75 - 2 = +1.609375 > 0$
- $f(a_2) \cdot f(c_2) = (-0.125) \cdot (1.609375) < 0 \implies$ Kök [1.5, 1.75] aralığındadır.
- Bağıl Hata: $\epsilon_a = \left| \frac{1.7500 - 1.5000}{1.7500} \right| \times 100\% = \%14.29$
- Yeni aralık: $a_3 = 1.5000, \ b_3 = 1.7500$

3. İterasyon (k=3):

- $c_3 = \frac{1.5 + 1.75}{2} = 1.6250$
- $f(c_3) = f(1.625) = (1.625)^3 - 1.625 - 2 = 4.291016 - 1.625 - 2 = +0.666016 > 0$
- $f(a_3) \cdot f(c_3) = (-0.125) \cdot (0.666016) < 0 \implies$ Kök [1.5, 1.625] aralığındadır.
- Bağıl Hata: $\epsilon_a = \left| \frac{1.6250 - 1.7500}{1.6250} \right| \times 100\% = \%7.69$
- Yeni aralık: $a_4 = 1.5000, \ b_4 = 1.6250$

4. İterasyon (k=4):

- $c_4 = \frac{1.5 + 1.625}{2} = 1.5625$
- $f(c_4) = f(1.5625) = (1.5625)^3 - 1.5625 - 2 = 3.814697 - 1.5625 - 2 = +0.252197 > 0$
- Bağıl Hata: $\epsilon_a = \left| \frac{1.5625 - 1.6250}{1.5625} \right| \times 100\% = \%4.00$

İterasyon Özeti Tablosu:

k	a_k	b_k	c_k	f(a_k)	f(b_
1	1.0000	2.0000	**1.5000**	-2.0000	+4.0
2	1.5000	2.0000	**1.7500**	-0.1250	+4.0
3	1.5000	1.7500	**1.6250**	-0.1250	+1.6
4	1.5000	1.6250	**1.5625**	-0.1250	+0.6

4 iterasyon sonucunda yaklaşık kök $x^* \approx \mathbf{1.5625}$'tir (Gerçek analitik kök ≈ 1.5214).

Örnek 1.2: Transandant (Trigonometrik) Denklem ve Durma Kriteri

Soru: $f(x) = x - \cos(x) = 0$ denkleminin $[0, 1]$ aralığındaki kökünü İkiye Bölme Yöntemi ile $\epsilon_a < \%1.0$ toleransı sağlanana kadar hesaplayınız (x radyan cinsindendir).

Çözüm:
Başlangıç kontrolü:

- $f(0) = 0 - \cos(0) = 0 - 1 = -1 < 0$
- $f(1) = 1 - \cos(1) = 1 - 0.540302 = +0.459698 > 0$
- $f(0) \cdot f(1) < 0 \implies$ Kök $[0, 1]$ aralığındadır.

İterasyon adımları:

1. **k=1:** $c_1 = \frac{0 + 1}{2} = 0.500000$, $f(0.5) = 0.5 - \cos(0.5) = 0.5 - 0.877583 = -0.377583 < 0 \implies [0.5, 1]$
2. **k=2:** $c_2 = \frac{0.5 + 1}{2} = 0.750000$, $f(0.75) = 0.75 - \cos(0.75) = +0.018311 > 0 \implies [0.5, 0.75]$
   - $\epsilon_a = \left|\frac{0.75 - 0.5}{0.75}\right| \times 100\% = 33.33\%$
3. **k=3:** $c_3 = \frac{0.5 + 0.75}{2} = 0.625000$, $f(0.625) = 0.625 - \cos(0.625) = -0.185964 < 0 \implies [0.625, 0.75]$
   - $\epsilon_a = \left|\frac{0.625 - 0.75}{0.625}\right| \times 100\% = 20.00\%$
4. **k=4:** $c_4 = \frac{0.625 + 0.75}{2} = 0.687500$, $f(0.6875) = -0.085335 < 0 \implies [0.6875, 0.75]$
   - $\epsilon_a = \left|\frac{0.6875 - 0.625}{0.6875}\right| \times 100\% = 9.09\%$
5. **k=5:** $c_5 = \frac{0.6875 + 0.75}{2} = 0.718750$, $f(0.71875) = -0.033787 < 0 \implies [0.71875, 0.75]$
   - $\epsilon_a = \left|\frac{0.71875 - 0.6875}{0.71875}\right| \times 100\% = 4.35\%$
6. **k=6:** $c_6 = \frac{0.71875 + 0.75}{2} = 0.734375$, $f(0.734375) = -0.007802 < 0 \implies [0.734375, 0.75]$

- \epsilon_a = \left|\frac{0.734375 - 0.71875}{0.734375}\right| \times 100\% = 2.13\%

7. **k=7:** c_7 = \frac{0.734375 + 0.75}{2} = 0.742188, f(0.742188) = +0.005238 > 0 \implies [0.734375, 0.742188]
   - \epsilon_a = \left|\frac{0.742188 - 0.734375}{0.742188}\right| \times 100\% = 1.05\%
8. **k=8:** c_8 = \frac{0.734375 + 0.742188}{2} = 0.738281
   - \epsilon_a = \left|\frac{0.738281 - 0.742188}{0.738281}\right| \times 100\% = \mathbf{0.53\% < 1.0\%} (Tolerans sağlandı!)

Yaklaşık kök x^* \approx \mathbf{0.738281} olarak bulunmuştur.

Örnek 1.3: Üstel Fonksiyon ve Teorik İterasyon Sayısı Hesabı

Soru: f(x) = e^{-x} - x = 0 denkleminin [0, 1] aralığında:

1. Mutlak hatanın \epsilon \le 10^{-2} = 0.01 olması için gereken **minimum iterasyon sayısını (n)** teorik olarak hesaplayınız.
2. İlk 3 iterasyonu gerçekleştirerek adım adım aralık daralmasını gösteriniz.

Çözüm:

1. İterasyon Sayısı Hesabı:
 n \ge \frac{\ln(b - a) - \ln(\epsilon)}{\ln 2} = \frac{\ln(1 - 0) - \ln(0.01)}{\ln 2} = \frac{0 - (-4.60517)}{0.693147} = \frac{4.60517}{0.693147} \approx 6.64İterasyon sayısı tam sayı olacağından en az **n = 7 iterasyon** yapılmalıdır.

2. İlk 3 İterasyon:

- Başlangıç: f(0) = e^0 - 0 = 1 > 0, f(1) = e^{-1} - 1 = 0.367879 - 1 = -0.632121 < 0.
- **k=1:** c_1 = \frac{0 + 1}{2} = 0.5000, f(0.5) = e^{-0.5} - 0.5 = 0.606531 - 0.5 = +0.106531 > 0 \implies [0.5, 1]
- **k=2:** c_2 = \frac{0.5 + 1}{2} = 0.7500, f(0.75) = e^{-0.75} - 0.75 = 0.472367 - 0.75 = -0.277633 < 0 \implies [0.5, 0.75]
- **k=3:** c_3 = \frac{0.5 + 0.75}{2} = 0.6250, f(0.625) = e^{-0.625} - 0.625 = 0.535261 - 0.625 = -0.089739 < 0 \implies [0.5, 0.625]

3 adım sonunda kökün bulunduğu aralık [0.5, 0.625] aralığına sıkıştırılmıştır ve yaklaşık kök c_3 = \mathbf{0.6250}'dir.

2. Sabit Nokta İterasyonu (Fixed-Point Iteration / Basit İterasyon)

Sabit nokta iterasyonu, açık (open) yöntemler sınıfına giren, bir denklemi x = g(x) formuna dönüştürerek ardışık yaklaşımlarla köke ulaşmayı amaçlayan genel bir tekniktir.

2.1. Yöntemin Mantığı ve Dönüşüm

f(x) = 0 denklemi cebirsel işlemlerle şu forma dönüştürülür:
x = g(x)Burada g(x^*) = x^* eşitliğini sağlayan x^* değerine g fonksiyonunun **sabit noktası (fixed point)** ve dolayısıyla f(x)=0 denkleminin **kökü** denir.

İterasyon formülü:
x_{k+1} = g(x_k), \quad k = 0, 1, 2, \dotsBaşlangıç için kullanıcı tarafından bir x_0 tahmini seçilir.

2.2. Yakınsaklık Şartı (Convergence Criterion)

Bir f(x)=0 denkleminden sonsuz sayıda farklı x=g(x) dönüşümü üretilebilir; ancak her dönüşüm köke yakınsamaz.

Sabit Nokta Teoremi (Banach):
x^* gerçek kökü içeren bir aralıkta g(x) ve g'(x) sürekli olsun:

- **Yakınsama Şartı:** Kök civarında |g'(x^*)| < 1 ise iterasyon x^* köküne **kesinlikle yakınsar**.
    - 0 < g'(x^*) < 1 \implies **Monoton Yakınsama** (değerler köke tek yönden yaklaşır).
    - -1 < g'(x^*) < 0 \implies **Salınımlı / Spiral Yakınsama** (değerler kökün etrafında zikzak çizerek yaklaşır).
- **Iraksama Durumu:** |g'(x^*)| > 1 ise iterasyon kökten uzaklaşır (**ıraksar**).
- |g'(x^*)| = 1 ise yakınsaklık garanti edilemez (kararsız).

|g'(x)| < 1 |g'(x)| > 1

_____

_____

KÖKE YAKINSAR (Converges) KÖKTEN IRAKSAR (Diverges)

Sabit Nokta İterasyonu — Çözümlü Örnekler

Örnek 2.1: Farklı g(x) Dönüşümlerinin Yakınsaklık Analizi

Soru: f(x) = x^3 + x - 1 = 0 denkleminin [0, 1] aralığında bir kökü bulunmaktadır (x^* \approx 0.6823).

1. Aşağıdaki iki farklı x = g(x) dönüşümünün yakınsaklık durumunu türev testiyle (|g'(x)| < 1) inceleyiniz:
   - g_1(x) = 1 - x^3
   - g_2(x) = \frac{1}{x^2 + 1}
2. Yakınsak olan dönüşümü kullanarak x_0 = 0.5 başlangıç değeriyle 4 iterasyon yapınız ve \epsilon_a değerlerini hesaplayınız.

Çözüm:

1. Yakınsaklık Analizi:

- **1. Dönüşüm:** x = 1 - x^3 \implies g_1(x) = 1 - x^3
  - Türev: g'_1(x) = -3x^2
  - Kök civarında (x^* \approx 0.6823):
    |g'_1(0.6823)| = |-3(0.6823)^2| = |-3(0.4655)| = 1.3965 > 1
  - **Sonuç:** |g'_1(x)| > 1 olduğundan bu dönüşüm **IRAKSAR**, kullanılamaz!
- **2. Dönüşüm:** x(x^2 + 1) = 1 \implies x = \frac{1}{x^2 + 1} \implies g_2(x) = \frac{1}{x^2 + 1}
  - Türev: g'_2(x) = \frac{-2x}{(x^2 + 1)^2}
  - Kök civarında (x^* \approx 0.6823):
    |g'_2(0.6823)| = \left| \frac{-2(0.6823)}{(0.6823^2 + 1)^2} \right| = \left| \frac{-1.3646}{(1.4655)^2} \right| = \frac{1.3646}{2.1477} \approx 0.6354 < 1
  - **Sonuç:** |g'_2(x)| < 1 olduğundan bu dönüşüm **YAKINSAR**.

2. İterasyon Adımları (x_{k+1} = \frac{1}{x_k^2 + 1}, x_0 = 0.5):

- **k=0:** x_0 = 0.500000
- **k=1:** x_1 = \frac{1}{0.5^2 + 1} = \frac{1}{1.25} = 0.800000

- \epsilon_a = \left|\frac{0.8000 - 0.5000}{0.8000}\right| \times 100\% = \%37.50

- **k=2:** x_2 = \frac{1}{0.8^2 + 1} = \frac{1}{0.64 + 1} = \frac{1}{1.64} = 0.609756
  - \epsilon_a = \left|\frac{0.609756 - 0.800000}{0.609756}\right| \times 100\% = \%31.20
- **k=3:** x_3 = \frac{1}{(0.609756)^2 + 1} = \frac{1}{0.371802 + 1} = \frac{1}{1.371802} = 0.728968
  - \epsilon_a = \left|\frac{0.728968 - 0.609756}{0.728968}\right| \times 100\% = \%16.35
- **k=4:** x_4 = \frac{1}{(0.728968)^2 + 1} = \frac{1}{0.531394 + 1} = \frac{1}{1.531394} = 0.653000
  - \epsilon_a = \left|\frac{0.653000 - 0.728968}{0.653000}\right| \times 100\% = \%11.63

İterasyonlar kök olan 0.6823 etrafında salınarak (zikzak çizerek) başarıyla yakınsamaktadır.

Örnek 2.2: Transandant Denklemde Hata Toleransına Göre İterasyon

Soru: f(x) = 2x - e^{-x} = 0 denklemini x = g(x) = \frac{1}{2}e^{-x} şeklinde düzenleyerek x_0 = 0 başlangıç değeriyle \epsilon_a < \%0.5 toleransı sağlanana kadar çözünüz.

Çözüm:

1. Yakınsaklık Kontrolü:
 g(x) = \frac{1}{2}e^{-x} \implies g'(x) = -\frac{1}{2}e^{-x}Pozitif x değerleri için |g'(x)| = \frac{1}{2}e^{-x} \le \frac{1}{2} < 1. Yöntem kesinlikle yakınsar.

2. İterasyonlar (x_{k+1} = 0.5 \cdot e^{-x_k}):

k	x_k	x_{k+1} = g(x_k)	$\epsilon_a = \left	\frac{x*{k+1} - x_k}{x*{k+1}} \right		
0	0.000000	0.500000	—	Başlangıç
1	0.500000	0.303265	64.87%	Devam
2	0.303265	0.369200	17.86%	Devam
3	0.369200	0.345644	6.81%	Devam

4	0.345644	0.353883	2.33%	Devam
5	0.353883	0.350978	0.83%	Devam
6	0.350978	**0.352000**	**0.29%**	\epsilon_a < \%0.5 (DUR)

1. adımda \epsilon_a = \%0.29 < \%0.50 şartı sağlanmış ve yaklaşık kök x^* \approx \mathbf{0.352000} olarak bulunmuştur.

Örnek 2.3: Karekök Hesabı ve Hızlı Yakınsayan Sabit Nokta Formu

Soru: \sqrt{5} sayısını bulmak f(x) = x^2 - 5 = 0 denklemini çözmek demektir.

1. x = g_A(x) = \frac{5}{x} dönüşümünün neden köke ulaşamadığını gösteriniz.
2. Denklemin her iki tarafına x eklenip 2'ye bölünmesiyle elde edilen x = g_B(x) = \frac{1}{2}\left(x + \frac{5}{x}\right) (Babylonian / Newton biçimi) dönüşümünün yakınsaklığını inceleyip x_0 = 2.0 başlangıç değeriyle 3 adımını çözünüz.

Çözüm:

1. g_A(x) = \frac{5}{x} İncelemesi:

- Türev: g'_A(x) = -\frac{5}{x^2}
- Gerçek kök x^* = \sqrt{5} noktasında:
  $$|g'_A(\sqrt{5})| = \left| -\frac{5}{(\sqrt{5})^2} \right| = \left| -\frac{5}{5} \right| = 1$$
- Türev 1 olduğu için sistem köke yaklaşamaz, iki değer arasında sonsuz döngüye girer:
  $$x_0 = 2 \implies x_1 = \frac{5}{2} = 2.5 \implies x_2 = \frac{5}{2.5} = 2.0 \implies x_3 = 2.5 \dots \text{(Döngü)}$$

2. g_B(x) = \frac{1}{2}\left(x + \frac{5}{x}\right) İncelemesi ve İterasyonları:

- Türev:
  $$g'_B(x) = \frac{1}{2}\left(1 - \frac{5}{x^2}\right)$$
- Kök noktasında ($x^* = \sqrt{5}$):
  $$g'_B(\sqrt{5}) = \frac{1}{2}\left(1 - \frac{5}{5}\right) = 0$$

- $|g'(x^*)| = 0$ olması yöntemin **ikinci dereceden (karesel - quadratic)** çok yüksek hızla yakınsayacağını gösterir!

İterasyon Adımları ($x_0 = 2.0$):

- **k=0:** $x_0 = 2.000000$
- **k=1:**
  $x_1 = \frac{1}{2}\left(2.0 + \frac{5}{2.0}\right) = \frac{1}{2}(2.0 + 2.5) = \mathbf{2.250000}\epsilon_a = \left|\frac{2.250000 - 2.000000}{2.250000}\right| \times 100\% = \%11.11$
- **k=2:**
  $x_2 = \frac{1}{2}\left(2.25 + \frac{5}{2.25}\right) = \frac{1}{2}(2.25 + 2.222222) = \mathbf{2.236111}\epsilon_a = \left|\frac{2.236111 - 2.250000}{2.236111}\right| \times 100\% = \%0.62$
- **k=3:**
  $x_3 = \frac{1}{2}\left(2.236111 + \frac{5}{2.236111}\right) = \frac{1}{2}(2.236111 + 2.236025) = \mathbf{2.236068}\epsilon_a = \left|\frac{2.236068 - 2.236111}{2.236068}\right| \times 100\% = \%0.0019$

Sadece 3 iterasyonda $\sqrt{5} \approx \mathbf{2.236068}$ gerçek değerine 6 ondalık basamak hassasiyetle ulaşılmıştır.